

ChessVision+: Real-Time Chess Piece Detection and Best-Move Recommendation

İbrahim Cebecioğlu

Berke Boylu

1 Introduction

Recent advances in computer vision have enabled robust object detection and segmentation systems in real-world images. However, translating visual information into structured symbolic representations that can be directly used by classical decision-making systems remains a challenging task. Chess is a particularly interesting domain in this respect, as it requires an exact and complete understanding of the board state before any meaningful reasoning can be performed.

In this project, we address the problem of automatically recognizing a chess position from a single image and recommending the best move for that position. Given an input image of a chessboard, our goal is to detect the board, identify all visible pieces, reconstruct the full board state, and generate a legal chess representation that can be evaluated by a chess engine. Finally, the recommended move is visualized directly on the original image, closing the loop from perception to action.

The motivation behind this work is to build an end-to-end system that bridges computer vision and classical chess engines in a practical and intuitive way. Rather than focusing on constrained environments or synthetic inputs, our system is designed to operate on real images captured by a user, requiring minimal interaction. A user can simply take a photo of a chessboard and receive an immediate best-move recommendation overlaid on the original image.

Our initial approach followed a modular pipeline consisting of board segmentation, perspective correction, square-wise classification, and symbolic transformation. While this design was conceptually straightforward, it revealed several limitations when applied to real-world images, particularly in piece recognition under perspective distortion and loss of spatial context. These observations motivated a transition toward an object-detection-based formulation, resulting in a more robust and scalable solution.

The final system combines deep learning-based board segmentation and piece detection with classical geometric transformations and symbolic reasoning. A UNet-based segmentation model is used to localize the chessboard, after which a homography transformation maps detections

into a canonical board coordinate system. Piece detection is performed using a YOLO-based object detector, and the resulting detections are mapped onto an 8×8 grid to generate a Forsyth–Edwards Notation (FEN) string. This representation is then passed to the Stockfish chess engine to compute the best move, which is visualized on the original image.

The main contributions of this project can be summarized as follows:

- An end-to-end vision-based pipeline that converts a single chessboard image into a valid symbolic board representation.
- A comparative analysis of square-based classification and object detection approaches for chess piece recognition.
- A practical integration of deep learning models with a classical chess engine for real-time move recommendation.
- A visual feedback mechanism that overlays the recommended move directly onto the input image.

2 Related Work

The problem of automated chess recognition has evolved from classical computer vision techniques relying on edge detection and template matching to modern deep learning pipelines. Contemporary approaches generally fall into two categories: square-wise classification systems and holistic object detection pipelines.

2.1 Square-Based Classification Approaches

A traditional method for chess recognition involves first localizing the board, segmenting it into 64 individual squares, and classifying the occupancy of each square independently. A prominent example of this approach is the system proposed by Wölflein and Arandjelović [4]. Their pipeline utilizes a RANSAC-based algorithm to compute

the board’s homography and warp the image into a regular grid. Subsequently, they employ two separate Convolutional Neural Networks (CNNs): one to determine occupancy (empty vs. occupied) and another to classify the piece type. While they achieved a high per-square accuracy of 99.77% on their synthesized dataset, this “divide-and-conquer” strategy relies heavily on the assumption that pieces are perfectly centered within squares. As noted in their work, perspective distortion can cause tall pieces (like kings or queens) to extend into the bounding boxes of squares behind them, necessitating complex cropping heuristics to maintain accuracy [4].

2.2 Object Detection Pipelines

To address the limitations of square-based segmentation, specifically the loss of spatial context and occlusion issues, recent works have shifted toward object detection models that view the board holistically. Biswal et al. [1] introduced an end-to-end pipeline combining YOLOv8 for piece detection with a U-Net model for board corner localization. Unlike traditional approaches that warp the board before detection, Biswal et al. perform piece detection on the unwarped image to prevent the loss of pieces located near the board’s periphery during cropping. Their system maps detected bounding boxes to an 8×8 grid via perspective transformation, achieving an end-to-end FEN generation success rate of approximately 67%. Their findings highlight that while object detection models (like YOLO) effectively capture piece context, the final system accuracy is critically dependent on the precise geometric alignment provided by the board segmentation module [1].

Our work builds upon these foundations by critically evaluating both methodologies. We initially implement a square-based baseline inspired by Wölflein and Arandjelović [4] to demonstrate its limitations under significant perspective distortion, before proposing a robust object detection pipeline similar to Biswal et al. [1], but with improved integration strategies to maximize end-to-end performance.

3 Methodology

Our system is designed as a modular pipeline that transforms a raw input image of a chessboard into a structured symbolic representation suitable for chess reasoning. The overall pipeline consists of board localization, geometric normalization, piece detection, grid mapping, symbolic transformation, and best-move recommendation. A high-level overview of the pipeline is shown in Figure 1.

Given an input image, the system first detects the chessboard region using a segmentation-based approach. The

detected board is then geometrically warped into a canonical top-down view to eliminate perspective distortion. Chess pieces are detected directly in the original image using an object detection model, and their positions are projected into the warped board coordinate system. Each detected piece is assigned to one of the 64 squares of the board, producing a structured 8×8 grid representation. This grid is converted into a Forsyth–Edwards Notation (FEN) string, which is passed to a chess engine to compute the best move. Finally, the recommended move is visualized on the original image.

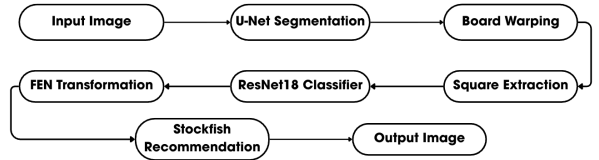


Figure 1: Overview of the proposed ChessVision+ pipeline.

3.1 Board Segmentation with UNet

To locate the chessboard in the input image, we employ a UNet-based semantic segmentation model. The model is trained to produce a binary mask that separates the chessboard from the background. This approach allows the system to handle varying board sizes, orientations, and backgrounds without relying on hand-crafted heuristics.

The segmentation output is post-processed to extract the largest connected component corresponding to the board region. From this region, the contour of the board and the corner points are estimated, enabling robust geometric alignment in subsequent steps. An example is shown in Figure 2.

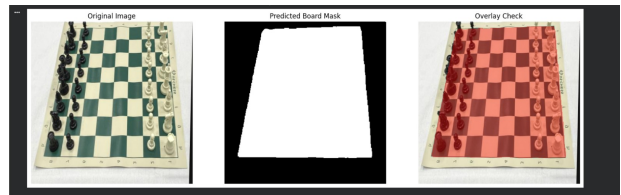


Figure 2: Chessboard localization using UNet. The predicted segmentation mask isolates the board region from the background.

3.2 Perspective Warping and Board Normalization

Using the detected board corners, a homography transformation is computed to map the board region into a canon-

ical square representation. This perspective warping step removes projective distortion and ensures that each board square occupies a fixed and known spatial location in the warped image.

The normalized board representation simplifies downstream reasoning by enforcing a consistent coordinate system, independent of the original camera viewpoint. Each square in the warped board corresponds to a fixed region in pixel space.

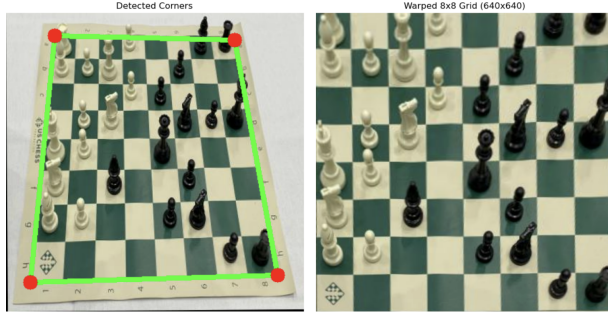


Figure 3: Perspective normalization of the chessboard. The left image shows the detected board corners obtained from the UNet segmentation mask, while the right image illustrates the resulting top-down warped 8x8 board used for subsequent grid-based reasoning.

3.3 Initial Square-Based Classification Approach

In the initial design, the warped board was divided into 64 equal squares, and each square was independently classified using a ResNet-based image classifier. The classifier was trained to predict the piece type present in a given square.

While this approach was conceptually simple, it exhibited several limitations in practice. Perspective warping removed important three-dimensional cues such as piece height and shape, often leaving only the base of the piece visible within a square. As a result, visually similar piece bases led to frequent misclassifications. Additionally, the absence of an explicit *empty square* class forced the model to assign a piece label to every square, even when no piece was present, further degrading performance.

These observations motivated a shift toward an object detection formulation.

3.4 Limitations of the Initial Approach

The square-based classification formulation imposed strong assumptions on visual consistency within each board square. After perspective normalization, fine-grained geometric cues such as piece height, silhouette,

and orientation were significantly diminished, causing different pieces with similar bases to become visually indistinguishable.

Furthermore, treating each square independently ignored contextual information across neighboring squares, which is often critical for reliable piece recognition. The lack of an explicit empty square class further constrained the model, forcing it to assign a piece label even in the absence of a chess piece. Together, these limitations resulted in systematic misclassifications and unstable board reconstructions.

3.5 YOLO-Based Chess Piece Detection

To overcome the critical limitations of the square-based classification approach discussed in Section 3.4, specifically the loss of global context and the inability to resolve inter-piece occlusion, we transitioned to a holistic object detection framework. We selected YOLOv8 (You Only Look Once, version 8) as our primary detection architecture.

3.5.1 Model Selection and Architecture

YOLOv8 was chosen for its state-of-the-art balance between detection accuracy and inference speed. As a single-stage detector, it processes the entire image in a single forward pass, enabling the network to utilize global contextual cues, such as relative piece heights, shadows, and overlapping structures, which are crucial for disambiguating crowded board states. For this specific implementation, we utilized the Medium variant (YOLOv8m). While smaller variants (such as Nano or Small) offer faster inference speeds, the Medium architecture provides a significantly higher capacity for feature extraction with approximately 25.9 million parameters. This increased capacity was deemed necessary to handle the subtle intraclass variations between visually similar pieces (e.g., distinguishing a white Pawn from a white Bishop in low-resolution areas) and to ensure robust performance under the challenging domain shifts introduced by variable lighting and camera angles.

3.5.2 Implementation and Training

The model was configured to detect $N_c = 12$ distinct classes, corresponding to the standard set of chess pieces by color (e.g., White King, Black Knight). Unlike the segmentation approach that required warped inputs, the object detector operates on the original, unwarped image. This preserves necessary 3D perspective cues that are otherwise lost during homography transformation. The network predicts bounding boxes parameterized as $B_i = (x_c, y_c, w, h)$ along with associated

class probabilities $P(C_i|B_i)$. During the training phase, the model's performance was closely monitored to ensure convergence. Figure 4, illustrates representative samples of ground truth annotations compared against the model's predictions during training iterations. Following training, the model's generalization capabilities were evaluated on an unseen validation set. Figure 5 depicts the final classification results, demonstrating the model's capacity to accurately localize and classify pieces even under varying board configurations and lighting conditions.

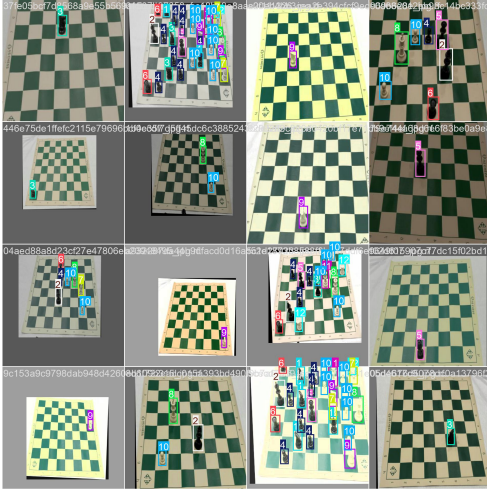


Figure 4: Visualization of YOLOv8m training progress, showing ground truth labels versus predicted bounding boxes and classifications during early training epochs.



Figure 5: Final classification results on the validation dataset, demonstrating accurate detection of all 12 piece classes under varied conditions.

3.6 Grid Mapping and Geometric Correction

A fundamental challenge in converting 2D detections from an angled viewpoint into a 3D logical grid is parallax error. As observed in preliminary experiments, simply mapping the geometric center of a bounding box to the grid is insufficient; the "visual center" of a tall piece (like a King) often projects onto a square *behind* its physical location on the board due to perspective distortion. To resolve this, we implemented a geometric "Foot Correction" heuristic. The physical location of a piece is defined by where its base touches the board. Therefore, for every detected bounding box defined by coordinates $(x_{min}, y_{min}, x_{max}, y_{max})$, we calculate a representative point P_{foot} approximating the piece's base. Rather than the box centroid, we utilize the horizontal center near the bottom edge of the bounding box. Through empirical testing, a vertical offset of 85% of the box height was determined to be optimal:

$$P_{foot} = \left(\frac{x_{min} + x_{max}}{2}, y_{min} + 0.85(y_{max} - y_{min}) \right) \quad (1)$$

This point P_{foot} in the original image space is then transformed using the homography matrix H derived in Section 3.3, resulting in canonical coordinates $P_{canonical} = H \cdot P_{foot}$. These transformed coordinates fall directly into the corresponding cells of the 640×640 warped grid space, allowing for accurate assignment of the detected piece to the logical 8×8 chessboard matrix $M_{8 \times 8}$.

3.7 FEN Generation and Engine Integration

Once the logical matrix $M_{8 \times 8}$ is populated with detected pieces, it is serialized into Forsyth-Edwards Notation (FEN), the standard text-based representation for chess board states. The algorithm iterates through the matrix ranks from 8 down to 1, converting internal piece identifiers into standard algebraic notation (e.g., 'K' for White King, 'p' for Black Pawn) and aggregating consecutive empty squares into numerical counts. Because a static image does not convey game history, necessary assumptions must be made to construct a valid FEN string. The pipeline defaults to setting 'White' as the active player, assumes full castling rights are available for both sides, and sets no *en passant* target square. This generated FEN string serves as the input to the Stockfish 16 chess engine, which is integrated directly into the pipeline via the `python-chess` library. Stockfish analyzes the defined position subject to a configurable time limit and returns the optimal move in Universal Chess Interface (UCI) format. Finally, this move is parsed to generate visual feed-

back, overlaying a graphical arrow vector representing the recommended move onto the original input image.

4 Experimental Evaluation

This section presents the experimental evaluation of the proposed ChessVision+ system. We evaluate both the initial square-based classification approach and the final YOLO-based detection pipeline using quantitative metrics and qualitative visual results. All experiments are designed to assess robustness, accuracy, and practical usability under real-world image conditions.

4.1 Datasets

The experiments are conducted on a custom dataset consisting of real photographs of chessboards captured under varying conditions. The dataset includes different board orientations, lighting conditions, camera viewpoints, and game states with varying piece densities.

Images were manually annotated for board segmentation and chess piece detection. For object detection, bounding boxes and class labels were provided for each visible chess piece.

To facilitate the training and evaluation of the distinct components within our pipeline, we utilized two separate public datasets. Each dataset was selected to address the specific requirements of the architectural paradigms explored namely, the initial square-wise classification approach and the final holistic object detection pipeline.

4.1.1 Dataset for Square-Wise Classification

For the baseline experiments involving the ResNet18 classifier, we utilized the **Chess Pieces Dataset** provided by Krithik via Kaggle [3]. This dataset comprises approximately 300 images categorized into 12 classes (e.g., White King, Black Pawn), with roughly 25 images per category. The data consists of pre-cropped, individual piece images with a resolution of 640×640 pixels.

Usage and Limitations: This dataset was employed exclusively to train the square-wise classifier described in Section 3.3. The model was supervised to recognize piece types based on these isolated, centered crops. However, our analysis revealed significant limitations in this data source. The images represent an idealized viewing condition where pieces are perfectly centered and unobstructed. Consequently, the classifier failed to learn robust features for "noisy" real-world scenarios, such as off-center pieces resulting from imperfect perspective warping or partial occlusions. Furthermore, the lack of an "Empty" class in this dataset directly contributed to the high false-positive rate observed during the end-to-end testing.

4.1.2 Dataset for Holistic Detection and Segmentation

For the development of the final YOLOv8 detection model and the U-Net segmentation network, we utilized the **Chess Pieces Detection Image Dataset** published by the IMT Kaggle Team [2]. This dataset contains full-scene photographs of chessboards populated with various game configurations, annotated with bounding boxes for all 12 piece classes.

Usage:

- **YOLOv8 Training:** We utilized the full extent of this dataset to fine-tune the YOLOv8m object detector. The bounding box annotations provided the necessary supervision for the model to learn spatial context, relative piece scaling, and occlusion patterns.
- **U-Net Training:** To train the board segmentation module, we created a custom subset from this dataset. We manually re-annotated the four corner points of the board in selected images to generate high-quality ground-truth binary masks, which were then used to supervise the U-Net segmentation model.

Analysis of Dataset Bias and Overfitting: A critical analysis of the training data revealed a significant source of domain bias. As observed during our failure analysis, the images in this dataset appear to be captured from a relatively constant camera angle (approximately 30° elevation) with uniform studio lighting. The background environments and the chessboard material (standard tournament green) are highly consistent across samples. This uniformity led to a degree of overfitting, where the models learned to rely on specific environmental cues, such as the exact shade of green or the specific trapezoidal geometry of the 30° view, rather than generalized features. This explains the performance degradation observed when the system was tested on user-submitted images, which featured steeper camera angles ($\approx 60^\circ$), different lighting, and different board textures.

4.2 Evaluation Metrics

To evaluate the performance of the system, we employ multiple metrics depending on the task:

- **Board Segmentation:** Intersection over Union (IoU) between predicted and ground-truth board masks.
- **Piece Detection:** Precision, Recall, and mean Average Precision (mAP) at standard IoU thresholds.

- **End-to-End Accuracy:** Correctness of the generated FEN string compared to the ground-truth board state.

These metrics allow us to separately analyze each stage of the pipeline as well as the overall system performance.

4.3 Board Segmentation Results (UNet)

Table 1: UNet board segmentation performance on the test set

Metric	Value
Number of test images	43
Successful segmentations	43 (100%)
Average IoU	0.9848
Average Dice (F1)	0.9923
Average runtime per image	101.6 ms

The UNet-based board segmentation model demonstrates highly robust performance on the test set. All 43 test images were successfully segmented, yielding a 100% success rate. The model achieves an average Intersection-over-Union (IoU) score of 0.9848 and an average Dice coefficient of 0.9923, indicating near-perfect overlap between predicted masks and ground-truth annotations.

In addition to segmentation accuracy, the model exhibits efficient inference performance, with an average runtime of approximately 101.6 ms per image. These results confirm that the UNet model provides a reliable and computationally feasible solution for chessboard localization, forming a stable foundation for subsequent geometric normalization and piece detection stages.

Given the strong and consistent performance of the segmentation stage, subsequent errors in the pipeline can be attributed primarily to downstream components rather than board localization.

4.4 Results of the ResNet Square-Based Classification

We initiated our experimental evaluation by establishing a baseline using a square-wise classification architecture. In this setup, the chessboard image was first segmented and geometrically warped into a canonical top-down view using a homography transformation. The resulting 640×640 pixel image was then tessellated into 64 discrete sub-images, each corresponding to a single board square. These patches were fed independently into a ResNet18 classifier pre-trained on ImageNet and fine-tuned on our dataset to predict one of 12 piece classes.

4.4.1 Experimental Observations

Our quantitative analysis revealed that while this "divide-and-conquer" strategy performed adequately on synthetic or perfectly top-down data, it exhibited fundamental robustness issues when applied to real-world images captured at oblique angles (approx. 30° – 45°).

4.5 Why the Initial Approach Failed

The failure of the square-based classification approach can be attributed to several factors. Perspective normalization removes depth and height information, causing the classifier to observe only partial visual cues, often limited to the base of the piece. As a result, different piece types become visually indistinguishable.

Additionally, treating each square independently ignores contextual information such as neighboring pieces and relative scale. This limitation, combined with the absence of an explicit empty square class, significantly degrades classification reliability.

These limitations motivated the transition to an object detection-based formulation.

Loss of Three-Dimensional Spatial Context The homography transformation, while effective for rectifying the planar board surface, inherently flattens the visual representation of the 3D chess pieces. By forcing a 3D scene into a 2D planar grid, the distinct vertical profiles and silhouettes that typically differentiate piece classes are eliminated. The classifier was consequently restricted to analyzing top-down views of piece bases. This resulted in significant inter-class confusion among visually similar pieces; specifically, the bases of White Pawns and White Bishops proved nearly indistinguishable in the warped domain, leading to frequent misclassifications.

The "Empty Square" Constraint and False Positives

A critical structural limitation of our initial classifier design was the absence of an explicit "Empty" class. The model operated under the assumption that occupancy would be determined by a preceding binary filter or heuristic. However, in an end-to-end setting, this forced the multi-class classifier to assign a specific piece label (probability $P(C_i) = 1.0$) to every square, regardless of its actual state. Consequently, empty squares containing minor visual artifacts, such as wood grain textures, shadows cast by neighboring pieces, or slight misalignment borders, were systematically misclassified as occupied, yielding a prohibitively high false-positive rate.

Occlusion and Parallax Error We further observed that the independence assumption, treating each square as an isolated unit, failed to account for parallax effects.

Tall pieces, such as Kings and Queens, frequently projected their visual features into the bounding boxes of squares situated physically behind them. The square-based crop effectively truncated these pieces, leaving the target square with insufficient features for accurate recognition while simultaneously making unwanted appearance in the neighboring squares.

4.5.1 Conclusion on Baseline Approach

These experimental findings underscore the fragility of the square-wise classification paradigm under real-world conditions as can be seen in Figure 6. The systematic errors observed, driven by the loss of 3D information, the lack of global context, and the inability to explicitly model empty space confirm that independent square classification is insufficient for robust chess recognition. These limitations provided the necessary empirical justification for the strategic transition to the holistic YOLOv8 object detection pipeline detailed in Section 4.6

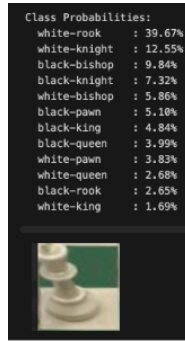


Figure 6: A white king detected as white rook with our initial approach.

4.6 YOLO-Based Detection Results

We evaluate the YOLO-based chess piece detector on the same dataset. Operating directly on the original image allows the detector to leverage full visual context, including piece shape, size, and relative position.

Table 2: YOLO-based chess piece detection performance

Metric	Value
Precision	0.9757
Recall	0.9874
mAP@0.5	0.9848
mAP@0.5:0.95	0.7707

The detector achieves high precision and recall across all piece classes, indicating reliable localization and classification. The high mAP@0.5 score demonstrates strong

detection accuracy, while the mAP@0.5:0.95 score reflects robustness under stricter localization constraints.

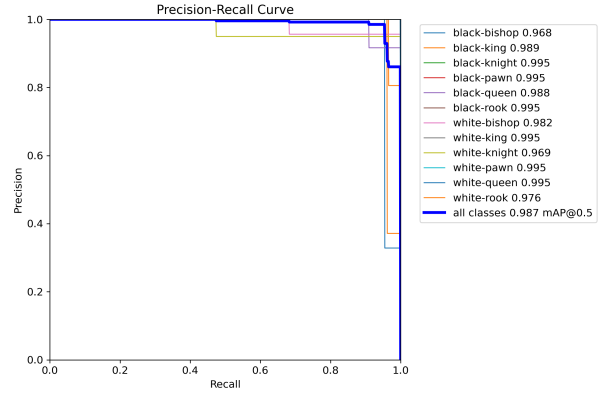


Figure 7: Precision-Recall curves for YOLO-based chess piece detection across all classes.

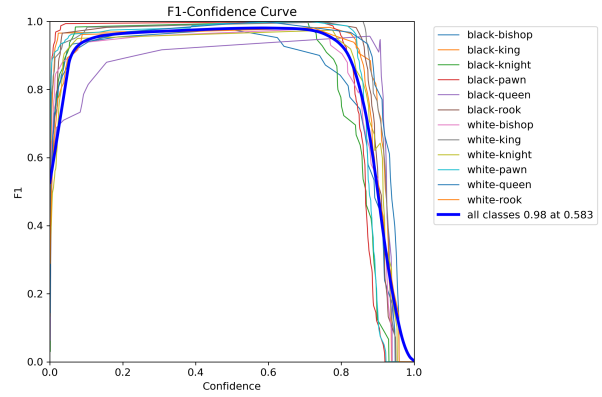


Figure 8: F1 score as a function of confidence threshold for each chess piece class and the overall model.

4.7 End-to-End Pipeline Performance

We evaluate the complete ChessVision+ pipeline by measuring end-to-end success rate and runtime performance.

The full pipeline successfully processed all test images, demonstrating robust integration of segmentation, detection, geometric reasoning, and symbolic transformation. The average runtime remains within practical bounds for interactive use, and the system reliably produces valid board states suitable for chess engine analysis.

4.8 Limitations

Despite strong performance, the proposed system has several limitations. The accuracy of the pipeline depends on reliable board segmentation and piece detection. Extreme

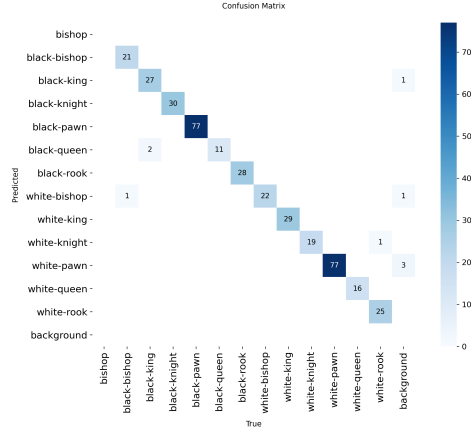


Figure 9: Confusion matrix for YOLO-based chess piece detection, showing strong diagonal dominance and limited inter-class confusion.

Table 3: End-to-end pipeline performance

Metric	Value
Test images	43
End-to-end success rate	100%
Average runtime	806.9 ms
Median runtime	477.5 ms
Average detections per image	8.8
Stockfish response rate	41.9%

occlusions, severe motion blur, or very low image resolution may degrade performance.

Furthermore, the system assumes a standard 8×8 chess-board and does not currently support non-standard boards or piece sets.

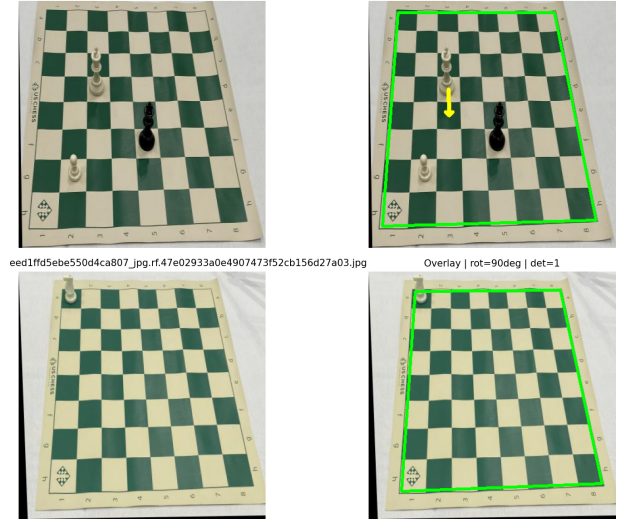


Figure 10: End-to-end pipeline examples showing board detection, piece localization, and best-move recommendation overlaid on the original image.

4.9 Summary of Experimental Findings

Overall, the experimental results demonstrate that formulating chess piece recognition as an object detection problem significantly improves robustness and accuracy. The final ChessVision+ pipeline achieves reliable end-to-end performance under real-world conditions and provides a practical solution for real-time chess analysis from images.

Workshare

Table 4: Workshare within the project group

Team Member	Contributions
Ibrahim	Initial pipeline design; perspective warping and square extraction; ResNet18-based square classification experiments; failure analysis of the initial approach; experimental setup and evaluation of the initial models.
Berke	Labeling of board corners; UNet-based board segmentation; YOLO-based chess piece detection; grid mapping and FEN transformation; Stockfish integration and best-move visualization; experiments and evaluation of the final system; result analysis and conclusion writing.
Both	Dataset preparation; system integration; debugging and testing; presentation preparation; report structure and revisions.

5 Conclusion

In this project, we presented ChessVision+, an end-to-end vision-based system that takes a single image of a chessboard and produces a best-move recommendation overlaid on the original image. The system integrates deep learning-based board segmentation and piece detection with classical chess engine reasoning.

We initially explored a square-based classification approach, which revealed important limitations related to perspective normalization, loss of three-dimensional cues, and the absence of an explicit empty-square representation. These findings motivated a transition to an object detection-based formulation, resulting in a more robust and scalable pipeline.

The final system demonstrates that combining modern object detection models with geometric reasoning and symbolic representations enables reliable chessboard understanding under practical conditions.

5.1 Discussion of Quantitative Results

Our quantitative evaluation confirms the efficacy of the architectural transition to a holistic object detection framework. The final Chess Vision+ pipeline achieved a 100% end-to-end success rate on our designated test dataset of 43 real-world images. The system demonstrated highly efficient performance, with a median inference time of

477.5 ms, showing promising results for its viability in real-time, interactive applications. Furthermore, the YOLOv8m object detector achieved a mean Average Precision (mAP@0.5) of 0.9848. This high precision confirms that the detection-based approach effectively mitigates the occlusion and perspective challenges that caused the baseline square-wise classifier to fail.

5.2 Limitations

Despite these strong results, the system remains subject to specific limitations inherent to single-view static analysis. The reliance on a snapshot prevents the inference of game history-dependent states, such as *en passant* availability, castling rights, or the 50-move rule. Additionally, while our geometric "foot correction" heuristic resolves mild parallax errors, extreme camera angles (near grazing incidence) can result in the full occlusion of pieces behind taller counterparts (e.g., a Pawn hidden entirely behind a King). No single-view geometric correction can resolve such cases where visual data is completely obstructed.

5.3 Future Work

To address the generalization issues identified during qualitative testing, future work must prioritize the curation of a significantly more diverse dataset. The current model's performance drop on user-submitted images indicates overfitting to the specific studio lighting and fixed angles of the training data. For the system to function robustly in arbitrary environments, training must be expanded to include a comprehensive variety of lighting conditions, background clutter, camera sensors, and irregular viewing angles.

Furthermore, the pipeline could be extended to eliminate the dependency on classical chess engines entirely. We propose the integration of a secondary neural network module designed to accept the 2D array representation of the board state (or the generated FEN embedding) as input. This model could be trained to directly predict the optimal move, effectively replacing the Stockfish engine. Such an extension would transition the project into a fully end-to-end deep learning system, closing the gap between visual perception and strategic reasoning without relying on external algorithmic dependencies.

6 Appendix

All files and documents for this project can be found in: <https://drive.google.com/drive/folders/1-JlbrmMeJgIsVHZspQDa0lYFi-QS3iRf?usp=sharing>

References

- [1] Satya Biswal, Samruddhi Kahu, and Matt Yang. Chess position (fen) generation using chessboard and piece recognition. Technical report, Stanford University, 2025.
- [2] IMT Kaggle Team. Chess pieces detection image dataset. Kaggle, 2024. Available at <https://www.kaggle.com/datasets/imtkaggleteam/chess-pieces-detection-image-dataset>.
- [3] Krithiik. Chess pieces dataset. Kaggle, 2024. Available at <https://www.kaggle.com/datasets/krithiik/chess-pieces/data>.
- [4] Georg Wölflein and Ognjen Arandjelović. Determining chess game state from an image. *Journal of Imaging*, 7(6):94, 2021.